

TECHNICAL ARTICLE & UPGRADE READINESS RUNBOOK

Getting Ready for PostgreSQL 19

Validated Features, Compatibility Risks, and DVD Rental Verification Scripts

Validation baseline: PostgreSQL 19 Beta 3 official documentation

Status date: 3 September 2026 / Pre-GA technical guide

Important release status PostgreSQL 19 is a beta release. Syntax, behavior, defaults, and release notes may change before general availability. Use this guide for lab testing and upgrade planning, not as approval to deploy PostgreSQL 19 Beta 3 in production.

Original work credited Inspired by “Getting Ready for PostgreSQL 19” by Dimitri Fontaine, published 3 September 2026 at tapoueh.org. The original article is credited in full; this document is an independently rewritten and expanded technical guide validated against PostgreSQL official documentation.

Table of Contents

1. Executive Summary	3
2. Release Status and Validation Method	3
3. Upgrade Compatibility Review	4
4. DVD Rental Lab Preparation	6
5. Temporal DML: FOR PORTION OF	6
6. INSERT ON CONFLICT DO SELECT	7
7. Window Functions: IGNORE NULLS	8
8. CHECK Constraint Enforcement	8
9. REPACK and REPACK CONCURRENTLY	9
10. DDL Reconstruction Functions	10
11. Range, Random, and NULL-Assertion Functions	11
12. SQL/PGQ Property Graphs	12
13. Logical Replication and Operations Highlights	13
14. Migration Runbook	14
15. Troubleshooting	15
16. Corrections and Deviations	16
17. Production Checklist	17
18. Credits and Official References	18

1. Executive Summary

PostgreSQL 19 Beta 3 introduces meaningful SQL, maintenance, replication, and observability changes. The most visible additions include SQL/PGQ property graphs, temporal UPDATE and DELETE with FOR PORTION OF, INSERT ... ON CONFLICT DO SELECT, IGNORE NULLS for selected window functions, the REPACK command, online checksum control, logical sequence replication, parallel autovacuum index work, and new catalog-to-DDL functions.

Main recommendation Treat PostgreSQL 19 as an evaluation target until GA. Build a repeatable DVD Rental test environment, inventory compatibility risks, run application and extension tests, compare performance, and repeat the complete test on the exact GA build before approving production migration.

Area	What changes	Administrator action
SQL development	Temporal DML, get-or-create, NULL-skipping windows, property graphs.	Test semantics, concurrency, privileges, and client compatibility.
Space maintenance	REPACK consolidates rewrite/cluster use cases; concurrent mode uses logical decoding.	Measure locks, disk/WAL, replication slots, DDL conflicts, and MVCC caveat.
Compatibility	JIT default, strings, RADIUS, MD5 warnings, GiST inet/cidr, lock sizing, TOAST default.	Resolve each item before pg_upgrade or cutover.
Replication	Sequences, effective WAL level, WAIT FOR, and publication changes.	Test failover/cutover and sequence continuity.
Operations	Online checksum changes, autovacuum priority/parallelism, new statistics.	Validate monitoring and update runbooks.

2. Release Status and Validation Method

The PostgreSQL project released PostgreSQL 19 Beta 3 on 13 August 2026. The version 19 release notes still show an unknown final release date and an “as of” date, so they are a working draft. Beta testing can confirm current behavior but cannot guarantee GA behavior.

- Primary authority: PostgreSQL 19 Beta 3 SQL-command, feature, configuration, and migration documentation.
- Secondary authority: PostgreSQL project release announcements and official PostgreSQL source-control history.

- Source article: used for topic selection and attribution, not as the final authority.
- DVDREntal scripts: objects are created in a separate pg19_lab schema; sample tables are read but not changed.
- Verification level: PostgreSQL 19-only grammar is checked against official syntax. It must be executed on the exact PostgreSQL 19 build under evaluation.
- Final gate: repeat all tests after GA and after the first selected minor release if production policy requires it.

Verification boundary This workspace does not contain a PostgreSQL 19 server binary. Therefore, the included DVDREntal scripts were checked for schema fit, dependency order, object qualification, rollback, and official PostgreSQL 19 grammar; PostgreSQL 19 runtime output must be captured in your own Beta 3 or later lab. Example timings and object sizes are intentionally not promised.

3. Upgrade Compatibility Review

3.1 Pre-upgrade inventory

```
SELECT version();
SHOW server_version_num;
SHOW jit;
SHOW standard_conforming_strings;
SHOW password_encryption;
SHOW max_locks_per_transaction;
SHOW default_toast_compression;

SELECT name, setting, source, pending_restart
FROM pg_settings
WHERE name IN ('jit','standard_conforming_strings','password_encryption',
               'max_locks_per_transaction','default_toast_compression','wal_level')
ORDER BY name;
```

3.2 Validated compatibility matrix

Change	Validated PostgreSQL 19 meaning	Required check
JIT default is off	New clusters default to disabled JIT. Existing configuration may override it.	Benchmark analytical SQL with JIT off/on; do not enable globally without evidence.
Standard strings always on	Legacy backslash-escape behavior cannot be restored server-side.	Use current pg_dump tools; inspect old applications and dumps created with off.
RADIUS removed	RADIUS authentication is no longer supported.	Replace matching pg_hba.conf entries before upgrade.

Change	Validated PostgreSQL 19 meaning	Required check
MD5 warnings	Successful MD5 password authentication can emit warnings; SCRAM is the migration target.	Inventory MD5 verifiers, clients, poolers, and channel binding support.
inet/cidr GiST opclass	Core GiST defaults replace affected btree_gist operator classes; unsafe old indexes block pg_upgrade.	Inventory and rebuild affected indexes as directed by the final migration notes.
Names with CR/LF	Database, role, and tablespace names cannot contain CR/LF.	Run catalog checks and rename invalid objects.
Lock sizing changed	Default max_locks_per_transaction is 128; migration notes require attention to old custom capacity.	Record old explicit value and follow the final PG19 sizing instruction.
TOAST default	Default is lz4 only when the build includes LZ4; otherwise pglz.	Check pg_config/build options and test CPU/storage. Existing values are not automatically rewritten.

TOAST correction Changing default_toast_compression affects compression chosen for newly stored compressible values. It is not an automatic cluster-wide rewrite of every existing TOAST value.

```
-- Find non-SCRAM password verifiers (superuser access required).
SELECT rolname,
       CASE WHEN rolpassword LIKE 'SCRAM-SHA-256$%' THEN 'SCRAM'
            WHEN rolpassword LIKE 'md5%' THEN 'MD5'
            WHEN rolpassword IS NULL THEN 'NO VERIFIER'
            ELSE 'OTHER' END AS verifier_type
FROM pg_authid
ORDER BY 1;

-- Find names containing carriage return or line feed.
SELECT 'database' AS object_type, datname AS object_name FROM pg_database
WHERE datname ~ E'[\r\n]'
UNION ALL SELECT 'role', rolname FROM pg_roles WHERE rolname ~ E'[\r\n]'
UNION ALL SELECT 'tablespace', spcname FROM pg_tablespace WHERE spcname ~ E'[\r\n]';

-- Review GiST indexes on inet/cidr columns; confirm opclass before action.
SELECT n.nspname, c.relname AS table_name, i.relname AS index_name,
       pg_get_indexdef(i.oid) AS index_definition
FROM pg_index x
JOIN pg_class i ON i.oid=x.indexrelid
JOIN pg_class c ON c.oid=x.indrelid
JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE i.relam=(SELECT oid FROM pg_am WHERE amname='gist')
AND pg_get_indexdef(i.oid) ~* '(inet|cidr)';
```

4. DVD Rental Lab Preparation

Connect to a restored DVD Rental database with PostgreSQL 19 Beta 3 or later. The scripts below assume the standard public.actor, film, customer, inventory, rental, payment, and category tables. They create only pg19_lab objects unless explicitly noted.

```
\set ON_ERROR_STOP on
SELECT current_database(), current_user, version();

DO $$
DECLARE missing text;
BEGIN
    SELECT string_agg(x, ', ' ORDER BY x) INTO missing
    FROM unnest(ARRAY['actor', 'film', 'customer', 'inventory',
                     'rental', 'payment', 'category']) AS x
    WHERE to_regclass('public.' || x) IS NULL;
    IF missing IS NOT NULL THEN
        RAISE EXCEPTION 'DVD Rental objects missing: %', missing;
    END IF;
END $$;

DROP SCHEMA IF EXISTS pg19_lab CASCADE;
CREATE SCHEMA pg19_lab;
SET search_path = pg19_lab, public, pg_catalog;

SELECT 'actor' AS table_name, count(*) FROM public.actor
UNION ALL SELECT 'film', count(*) FROM public.film
UNION ALL SELECT 'customer', count(*) FROM public.customer
UNION ALL SELECT 'rental', count(*) FROM public.rental
UNION ALL SELECT 'payment', count(*) FROM public.payment;
```

Safety Run the lab in a disposable copy. REPACK, authentication, checksums, logical replication, and upgrade tests can consume disk, WAL, locks, and replication resources. Never test these first on production.

5. Temporal DML: FOR PORTION OF

FOR PORTION OF lets UPDATE or DELETE target only the overlapping part of a range or multirange. PostgreSQL may shrink the affected row and insert zero, one, or two temporal leftovers. INSERT triggers can fire for leftovers, and READ COMMITTED concurrency needs special care; serialize changes to the same business key or use a stronger isolation/locking design.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE pg19_lab.customer_membership (
    customer_id smallint NOT NULL REFERENCES public.customer(customer_id),
    tier text NOT NULL,
    valid_period daterange NOT NULL,
    EXCLUDE USING gist (customer_id WITH =, valid_period WITH &&)
);

INSERT INTO pg19_lab.customer_membership
SELECT customer_id, 'STANDARD', daterange(DATE '2005-01-01', DATE '2007-01-01', '[]')
FROM public.customer ORDER BY customer_id LIMIT 1;
```

```

UPDATE pg19_lab.customer_membership
FOR PORTION OF valid_period FROM DATE '2006-01-01' TO DATE '2006-07-01'
SET tier = 'GOLD';

SELECT customer_id, tier, valid_period
FROM pg19_lab.customer_membership ORDER BY valid_period;

DELETE FROM pg19_lab.customer_membership
FOR PORTION OF valid_period FROM DATE '2006-02-01' TO DATE '2006-03-01'
WHERE tier = 'GOLD';

```

Expected shape The UPDATE can produce three non-overlapping periods: STANDARD before, GOLD during, and STANDARD after. The DELETE removes only the requested slice and retains the remaining history. Validate trigger, RLS, exclusion, and concurrent-update behavior in your application design.

6. INSERT ... ON CONFLICT DO SELECT

The new conflict action implements an atomic insert-or-select pattern and can optionally lock the conflicting row. A conflict target and RETURNING clause are required. Privileges and row-lock behavior still matter; this is not a substitute for transaction design.

```

CREATE TABLE pg19_lab.customer_registry (
  email text PRIMARY KEY,
  customer_id smallint NOT NULL REFERENCES public.customer(customer_id),
  first_seen_at timestamptz NOT NULL DEFAULT clock_timestamp()
);

INSERT INTO pg19_lab.customer_registry(email, customer_id)
SELECT email, customer_id FROM public.customer ORDER BY customer_id LIMIT 1
ON CONFLICT (email) DO SELECT
RETURNING email, customer_id, first_seen_at;

-- Run again: returns the existing row without changing first_seen_at.
INSERT INTO pg19_lab.customer_registry(email, customer_id)
SELECT email, customer_id FROM public.customer ORDER BY customer_id LIMIT 1
ON CONFLICT (email) DO SELECT FOR KEY SHARE
RETURNING email, customer_id, first_seen_at;

```

Action	Conflict result	Use
DO NOTHING	No row returned for the conflict unless separately queried.	Fire-and-forget deduplication.
DO UPDATE	Updates and can return the row.	True upsert when data must change.
DO SELECT	Returns existing row and can apply a row lock.	Idempotent get-or-create without a second round trip.

7. Window Functions: IGNORE NULLS

PostgreSQL 19 adds IGNORE NULLS and RESPECT NULLS to lead, lag, first_value, last_value, and nth_value. IGNORE NULLS skips null input values; window ordering and frame rules still control the answer.

```
WITH daily AS (
  SELECT d::date AS day, sum(p.amount)::numeric(10,2) AS revenue
  FROM generate_series(DATE '2005-05-24', DATE '2005-06-05', interval '1 day') AS g(d)
  LEFT JOIN public.payment p ON p.payment_date::date = d::date
  GROUP BY d::date
)
SELECT day, revenue,
       lag(revenue) OVER (ORDER BY day) AS previous_calendar_day,
       lag(revenue) IGNORE NULLS OVER (ORDER BY day) AS previous_revenue_day
FROM daily ORDER BY day;

-- Frame-aware last known value through the current row.
SELECT day, revenue,
       last_value(revenue) IGNORE NULLS OVER (
         ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS revenue_carried_forward
FROM daily ORDER BY day;
```

Plan carefully IGNORE NULLS does not invent values, change the sort order, or ignore NULL ordering rules. For last_value and nth_value, specify the intended window frame explicitly; the default frame often surprises users.

8. CHECK Constraint Enforcement

NOT VALID and NOT ENFORCED solve different problems. NOT VALID skips the initial scan of old rows but continues enforcing new changes. NOT ENFORCED records the rule but does not enforce it. PostgreSQL 19 can change enforcement for a CHECK constraint with ALTER TABLE ... ALTER CONSTRAINT.

```
CREATE TABLE pg19_lab.film_quality (
  film_id integer PRIMARY KEY REFERENCES public.film(film_id),
  quality_score integer,
  CONSTRAINT quality_score_check CHECK (quality_score BETWEEN 0 AND 100)
);
INSERT INTO pg19_lab.film_quality VALUES (1, 90);

ALTER TABLE pg19_lab.film_quality
  ALTER CONSTRAINT quality_score_check NOT ENFORCED;
INSERT INTO pg19_lab.film_quality VALUES (2, -5); -- accepted while disabled

-- This should fail until invalid rows are corrected.
ALTER TABLE pg19_lab.film_quality
  ALTER CONSTRAINT quality_score_check ENFORCED;

UPDATE pg19_lab.film_quality SET quality_score = 0 WHERE quality_score < 0;
ALTER TABLE pg19_lab.film_quality
  ALTER CONSTRAINT quality_score_check ENFORCED;
```


Production warning A NOT ENFORCED CHECK is documentation, not protection. Monitoring and data-quality tools must understand the enforcement state. Prefer NOT VALID plus VALIDATE CONSTRAINT when the real goal is an online backlog cleanup.

9. REPACK and REPACK CONCURRENTLY

REPACK rewrites table and index files to remove dead-tuple space and can order rows by an index. Without concurrent mode it takes ACCESS EXCLUSIVE for the operation. Concurrent mode captures DML through logical decoding and normally limits ACCESS EXCLUSIVE to the final swap, but it is explicitly not MVCC-safe and has several eligibility restrictions.

```
CREATE TABLE pg19_lab.payment_repack AS TABLE public.payment;
ALTER TABLE pg19_lab.payment_repack ADD PRIMARY KEY (payment_id);
DELETE FROM pg19_lab.payment_repack WHERE payment_id % 3 = 0;

SELECT pg_size_pretty(pg_total_relation_size('pg19_lab.payment_repack')) AS before;
REPACK (ANALYZE, VERBOSE) pg19_lab.payment_repack;
SELECT pg_size_pretty(pg_total_relation_size('pg19_lab.payment_repack')) AS after;

-- Concurrent form: do not put inside BEGIN/COMMIT.
REPACK (CONCURRENTLY, ANALYZE, VERBOSE) pg19_lab.payment_repack;

SELECT pid, relid::regclass, command, phase,
       heap_blks_total, heap_blks_scanned, heap_tuples_written
FROM pg_stat_progress_repack;
```

Syntax correction The documented concurrent syntax is REPACK (CONCURRENTLY) table_name, not “REPACK CONCURRENTLY table_name”. Use the exact grammar from the PostgreSQL 19 documentation.

Check	Why
Eligible relation	Concurrent mode rejects unlogged, partitioned, catalog, TOAST, and transaction-block cases.
Row identity	A primary key or index-based replica identity is required.
Replication slot capacity	max_repack_replication_slots must permit another slot.
Disk and WAL headroom	The rewrite creates new table/index files plus concurrent-change storage.
Long transactions and DDL	They can delay the final lock or cause the operation to fail.
MVCC behavior	Official documentation warns concurrent REPACK is not MVCC-safe.

Check	Why
Statistics	Use ANALYZE or run it afterward where appropriate.

10. DDL Reconstruction Functions

`pg_get_role_ddl()`, `pg_get_tablespace_ddl()`, and `pg_get_database_ddl()` reconstruct executable statements from catalogs. They return SETOF text. Output is decompiled, not the exact original input, and role passwords are never included. These functions supplement rather than automatically replace a full migration toolchain.

```
SELECT * FROM pg_get_database_ddl(current_database()::regdatabase);
SELECT * FROM pg_get_role_ddl(current_user::regrole);

SELECT spcname, ddl
FROM pg_tablespace t
CROSS JOIN LATERAL pg_get_tablespace_ddl(t.oid) AS x(ddl)
WHERE spcname NOT IN ('pg_default', 'pg_global');

-- Save ordered output with psql if required:
\pset tuples_only on
\pset format unaligned
SELECT * FROM pg_get_database_ddl(current_database()::regdatabase);
```

Security note Review generated role membership and ownership statements before execution. Password verifiers are deliberately absent, and tablespace filesystem paths may not be valid on the destination host.

11. Range, Random, and NULL-Assertion Functions

11.1 Range subtraction

```
SELECT *
FROM range_minus_multi(
    daterange(DATE '2005-05-24', DATE '2005-08-24', '[')'),
    daterange(DATE '2005-06-01', DATE '2005-07-01', '[')')
);

SELECT *
FROM multirange_minus_multi(
    datemultirange(daterange(DATE '2005-05-24', DATE '2005-08-24', '[')'),
    daterange(DATE '2005-06-01', DATE '2005-07-01', '[')')
);
```

`range_minus_multi` returns each surviving range fragment as a row. It is useful when subtracting a middle period that cannot be represented by one plain range. `multirange_minus_multi` works with a `multirange` result model.

11.2 Random dates and timestamps

```
SELECT setseed(0.42);
SELECT random(min(rental_date)::timestamp,
              max(rental_date)::timestamp) AS sample_rental_time
FROM public.rental
CROSS JOIN generate_series(1,5);

SELECT random(DATE '2005-05-24', DATE '2006-02-14') AS sample_day
FROM generate_series(1,5);
```

The PostgreSQL 19 overloads remove manual date arithmetic from test-data generation. `setseed` makes a sequence repeatable within the session; it is for reproducible tests, not security tokens or cryptographic randomness.

11.3 `error_on_null()`

```
-- Deliberate failure if the lookup returns no row.
SELECT error_on_null((
  SELECT actor_id FROM public.actor
  WHERE first_name='PENELOPE' AND last_name='GUINNESS'
)) AS actor_id;

-- A scalar subquery returning multiple rows still fails before error_on_null.
SELECT error_on_null((
  SELECT customer_id FROM public.customer WHERE email='NOT_FOUND@EXAMPLE.COM'
)) AS customer_id;
```

Correct scope `error_on_null` changes a silent scalar-subquery NULL into an error. It does not handle “more than one row returned” and does not replace unique constraints or cardinality validation.

12. SQL/PGQ Property Graphs

A PostgreSQL property graph is a read-only logical view over relational tables, views, or similar objects. It does not copy the data into a new storage engine. `GRAPH_TABLE` turns a graph pattern into a relation that can be joined with ordinary SQL. Users need privileges on both the graph and its base relations.

12.1 Build a DVD Rental graph

```
CREATE TABLE pg19_lab.customer_film_edge (
  rental_id integer PRIMARY KEY,
  customer_id smallint NOT NULL REFERENCES public.customer(customer_id),
  film_id smallint NOT NULL REFERENCES public.film(film_id),
  rental_date timestampz NOT NULL
);

INSERT INTO pg19_lab.customer_film_edge
SELECT r.rental_id, r.customer_id, i.film_id, r.rental_date
FROM public.rental r
JOIN public.inventory i USING (inventory_id);

CREATE PROPERTY GRAPH pg19_lab.dvdrental_graph
  VERTEX TABLES (
```

```

public.customer KEY (customer_id) LABEL customer
  PROPERTIES (first_name, last_name, email),
public.film KEY (film_id) LABEL film
  PROPERTIES (title, rating, rental_rate)
)
EDGE TABLES (
  pg19_lab.customer_film_edge KEY (rental_id)
    SOURCE KEY (customer_id) REFERENCES customer(customer_id)
    DESTINATION KEY (film_id) REFERENCES film(film_id)
    LABEL rented PROPERTIES (rental_date)
);

```

12.2 Query one-hop patterns

```

SELECT customer_name, film_title, rented_at
FROM GRAPH_TABLE (pg19_lab.dvdrental_graph
  MATCH (c IS customer)-[r IS rented]->(f IS film)
  COLUMNS (
    c.first_name || ' ' || c.last_name AS customer_name,
    f.title AS film_title,
    r.rental_date AS rented_at
  )
)
ORDER BY rented_at DESC
FETCH FIRST 20 ROWS ONLY;

```

Beta 3 limitation Do not assume full graph-database path functionality. In the documented Beta 3 state, variable-length element-pattern quantifiers and related shortest-path features are not complete. Use explicit fixed-hop patterns or WITH RECURSIVE where appropriate, then retest GA.

Property graph gives you	Property graph does not give you
Standard graph-pattern syntax over existing relational data.	A separate native graph storage engine.
A logical graph definition and GRAPH_TABLE relational output.	Free maintenance: underlying edge tables still need normal design.
Reuse of PostgreSQL planning/execution and base-table permissions.	Automatic support for every SQL/PGQ feature in Beta 3.
A clearer expression for some fixed relationship patterns.	A guarantee that graph syntax is faster than joins.

13. Logical Replication and Operations Highlights

Feature	What to test before adoption
---------	------------------------------

Feature	What to test before adoption
Sequence replication	Publication syntax, refresh behavior, sequence ownership, failover values, and application caching.
Effective WAL level	Activation timing, slot creation, WAL volume, restart expectations, and <code>effective_wal_level</code> monitoring.
WAIT FOR	Read-your-writes routing, timeouts, replay lag, standby failure, and connection-pool behavior.
Publication exclusions	Membership drift, schema changes, refresh behavior, and disaster-recovery rebuilds.
Online checksum control	I/O overhead, progress, failure handling, backup compatibility, and rollback procedure.
Parallel autovacuum	Index-phase gains, worker capacity, CPU/I/O contention; default max parallel workers is zero.
Autovacuum scoring	Alerting from <code>pg_stat_autovacuum_scores</code> and tuning against real churn/freeze risk.
Adaptive I/O workers	New min/max worker behavior versus the PostgreSQL 18 worker configuration.
New statistics/logging	Dashboard compatibility, retention, log volume, and alert thresholds.
Server-side SNI	Certificate selection, client SNI support, reload behavior, and fallback certificate.

```
-- PostgreSQL 19 readiness inventory (run only where objects exist).
SHOW wal_level;
SHOW effective_wal_level;
SHOW autovacuum_max_parallel_workers;
SELECT * FROM pg_stat_autovacuum_scores ORDER BY score DESC LIMIT 20;
SELECT * FROM pg_stat_recovery;
SELECT * FROM pg_stat_lock;
SELECT * FROM pg_get_multixact_stats();
```

Do not copy blindly Some PostgreSQL 19 views and functions can change during beta. Confirm column names with `\d+` or the exact documentation shipped with your build before integrating monitoring.

14. Migration Runbook

1. Record the source major/minor version, operating system, architecture, extensions, drivers, poolers, backup tools, replication topology, and recovery objectives.

2. Read the final PostgreSQL 19 release notes and migration section. Re-run the compatibility inventory; do not rely on a beta-era checklist alone.
3. Create a separate PostgreSQL 19 cluster. Never overwrite the existing major-version binaries or data directory.
4. Restore a fresh DVD Rental copy and run every lab section with `ON_ERROR_STOP`, recording outputs and deviations.
5. Test `pg_upgrade --check` or a dump/restore/logical-replication migration method that matches downtime and platform constraints.
6. Run application regression tests: authentication, prepared statements, transactions, DDL, drivers, ORM, extensions, encoding, and time zones.
7. Benchmark representative reads/writes with the same data volume, statistics, cache state, concurrency, and settings. Compare plans, buffers, WAL, CPU, I/O, and latency.
8. Exercise backup, restore, PITR, replica rebuild, failover, switchover, logical cutover, and sequence continuity.
9. Prepare cutover, validation, rollback, monitoring, ownership, and communication plans with explicit go/no-go criteria.
10. Repeat the complete rehearsal on the exact GA minor version and package build selected for production.

14.1 Suggested acceptance criteria

Gate	Example evidence
Compatibility	No unresolved <code>pg_upgrade</code> checks; extensions and tools support PG19.
Correctness	DVD Rental row counts, constraints, functions, application tests, and checksums pass.
Performance	Critical workload remains inside agreed latency/resource limits.
Recoverability	Independent restore and PITR complete inside RTO/RPO.
Replication	Lag, conflict, sequence, failover, and reconnect tests pass.
Operations	Dashboards, logs, alerts, jobs, backups, and on-call runbooks work.
Rollback	Decision deadline and tested reversal method are documented.

15. Troubleshooting

Symptom	Likely cause / check	Safe response
---------	----------------------	---------------

Symptom	Likely cause / check	Safe response
PG19 syntax error	Server is not PG19 or feature changed after Beta 3.	Check SELECT version(); use docs shipped with exact build.
Temporal DML creates extra rows	Expected temporal leftovers preserve untargeted periods.	Review ranges, triggers, RLS, and exclusion constraints.
DO SELECT returns no conflict row	WHERE filtered it, privileges/RLS, or no matching arbiter.	Inspect target, unique index, WHERE, RETURNING, and policies.
IGNORE NULLS looks wrong	ORDER BY or default frame differs from intent.	Specify deterministic order and explicit ROWS frame.
Cannot enforce CHECK	Existing data violates it.	Find and repair violations, then enable enforcement.
REPACK concurrent rejected	Unsupported relation, transaction block, identity, or slot limit.	Fix eligibility; use planned non-concurrent rewrite if approved.
REPACK waits at final stage	ACCESS EXCLUSIVE swap conflicts with transactions/DDDL.	Identify blockers; cancel only with owner approval; reschedule.
REPACK disk/WAL surge	Full rewrite plus indexes and captured DML.	Stop safely if needed; increase headroom; reduce churn/window.
Graph privilege error	Graph and/or base relation privilege missing.	Grant minimum required privileges; retest as application role.
Variable-length graph error	Feature not implemented in tested build.	Use WITH RECURSIVE or fixed hops; recheck GA.
pg_upgrade refuses cluster	Known compatibility issue or unsupported extension/object.	Follow exact report and final migration notes; never bypass checks.
Performance differs from article	Data, hardware, cache, build, settings, and concurrency differ.	Measure locally; retain EXPLAIN and system metrics.

```
-- General blocker view for maintenance operations.
SELECT a.pid, a.username, a.state, a.xact_start,
       pg_blocking_pids(a.pid) AS blocked_by,
       left(a.query, 160) AS query
FROM pg_stat_activity a
WHERE a.datname = current_database()
ORDER BY a.xact_start NULLS LAST;

SELECT pid, relid::regclass, command, phase
FROM pg_stat_progress_repack;

SELECT slot_name, slot_type, active, restart_lsn,
```

```
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;
```

16. Corrections and Deviations from the Source Article

Source wording / issue	Validated clarification
GA expected in “next few weeks.”	No GA date was announced as of 3 September 2026; planning estimates are not commitments.
Release notes imply MERGE/SPLIT PARTITIONS ships.	The feature was reverted from the PostgreSQL 19 branch on 27 August 2026. Beta 3 still contains it; later builds do not. Do not depend on it.
REPACK CONCURRENTLY table_name.	Official SQL grammar uses REPACK (CONCURRENTLY) table_name.
REPACK replaces VACUUM FULL and CLUSTER.	It unifies their rewrite/ordering use cases and old commands remain for compatibility. Locking, resources, eligibility, and semantics must still be evaluated.
TOAST default silently changes cluster-wide compression.	The default affects newly stored compressible values; existing values are not automatically rewritten. LZ4 depends on build support.
One temporal UPDATE simply produces two rows.	Depending on overlap, a range operation may create zero, one, or two leftovers, plus affected row versions; triggers and concurrency matter.
Property graph “costs nothing to maintain.”	The graph definition is not materialized, but base vertex/edge tables, indexes, statistics, permissions, and refresh logic still have operational cost.
DDL functions replace external dump tools.	They are useful reconstruction helpers but do not replace a complete, tested migration/backup process.
Every printed Beta 3 result reproduces universally.	Results depend on exact beta build, dataset, locale, seed, configuration, and later beta changes. Syntax must be retested on the target build.

17. Production Readiness Checklist

- ☐ PostgreSQL 19 GA release and selected minor/package build are available and approved.
- ☐ Final release notes and migration instructions have been reviewed after GA.
- ☐ Source configuration and all compatibility-breaking objects are inventoried.
- ☐ MD5/RADIUS dependencies and client authentication compatibility are resolved.
- ☐ Affected inet/cidr GiST indexes and invalid object names are resolved.
- ☐ Lock-memory and TOAST compression changes are capacity/performance tested.

- ☐ DVD Rental verification script completes with expected results on the target build.
- ☐ Extensions, drivers, poolers, monitoring, backup, and automation tools support PG19.
- ☐ REPACK capacity, locking, logical-decoding, DDL, and rollback tests pass.
- ☐ Temporal DML trigger, RLS, constraint, and concurrency behavior is accepted.
- ☐ Graph permissions and supported pattern scope are documented.
- ☐ Logical replication sequence and cutover behavior is proven.
- ☐ Backup, restore, PITR, failover, and rollback rehearsals meet RTO/RPO.
- ☐ Representative performance results meet agreed service objectives.
- ☐ Change approval, communication, monitoring, and post-cutover ownership are assigned.

Go/no-go rule A successful beta lab is evidence for planning only. Production approval requires a clean rehearsal on the exact PostgreSQL 19 GA build, supported ecosystem components, recoverability proof, and an agreed rollback path.

18. Credits and Official References

Original article credit: Dimitri Fontaine, “Getting Ready for PostgreSQL 19,” published on tapoueh.org on 3 September 2026. Credit belongs to the original author for the source article’s selection, examples, and commentary. This rewritten guide adds independent PostgreSQL official-documentation validation, DVD Rental-focused examples, operational cautions, migration steps, and corrections.

- Original article - Dimitri Fontaine: <https://tapoueh.org/blog/2026/09/getting-ready-for-postgresql-19/>
- PostgreSQL 19 Beta 3 announcement: <https://www.postgresql.org/about/news/postgresql-186-1711-1615-1519-1424-and-19-beta-3-released-3365/>
- PostgreSQL 19 release notes (draft): <https://www.postgresql.org/docs/19/release-19.html>
- Migration to PostgreSQL 19: <https://www.postgresql.org/docs/19/release-19.html#RELEASE-19-MIGRATION>
- UPDATE / FOR PORTION OF: <https://www.postgresql.org/docs/19/sql-update.html>
- DELETE / FOR PORTION OF: <https://www.postgresql.org/docs/19/sql-delete.html>
- Temporal data modification: <https://www.postgresql.org/docs/19/dml-application-time-update-delete.html>
- INSERT / ON CONFLICT DO SELECT: <https://www.postgresql.org/docs/19/sql-insert.html>
- Window functions: <https://www.postgresql.org/docs/19/functions-window.html>
- ALTER TABLE constraints: <https://www.postgresql.org/docs/19/sql-altertable.html>
- REPACK: <https://www.postgresql.org/docs/19/sql-repack.html>
- System information / DDL functions: <https://www.postgresql.org/docs/19/functions-info.html>
- Range and multirange functions: <https://www.postgresql.org/docs/19/functions-range.html>
- Comparison functions / error_on_null: <https://www.postgresql.org/docs/19/functions-comparison.html>
- Mathematical functions / random overloads: <https://www.postgresql.org/docs/19/functions-math.html>
- Property graphs: <https://www.postgresql.org/docs/19/ddl-property-graphs.html>

- **CREATE PROPERTY GRAPH:** <https://www.postgresql.org/docs/19/sql-create-property-graph.html>
- **GRAPH_TABLE:** <https://www.postgresql.org/docs/19/queries-table-expressions.html#QUERIES-GRAPH-TABLE>
- **Server configuration:** <https://www.postgresql.org/docs/19/runtime-config.html>
- **Official PostgreSQL source commit reverting MERGE/SPLIT:** <https://git.postgresql.org/gitweb/?p=postgresql.git;a=commit;h=3e8bcc864>

Reference discipline The PostgreSQL 19 documentation is pre-release documentation. Reopen every cited page after GA because anchors, syntax, defaults, limitations, and release-note entries may change.

```
-- Optional lab cleanup after evidence is saved.  
DROP PROPERTY GRAPH IF EXISTS pg19_lab.dvdrental_graph;  
DROP SCHEMA IF EXISTS pg19_lab CASCADE;
```